CAAITE · TRUNG TÂM ĐÀO TẠO AI CHUYÊN SÂU

Buổi 3: API và tích hợp LLM

Từ mô hình Client-Server đến việc tích hợp Gemini Flash bằng FastAPI

Người thuyết trình
Họ và tên · Chức danh

Mục tiêu buổi học

01

Nền tảng

- Mô hình Client-Server
- Giao thức HTTP
- Định dạng dữ liệu JSON

02

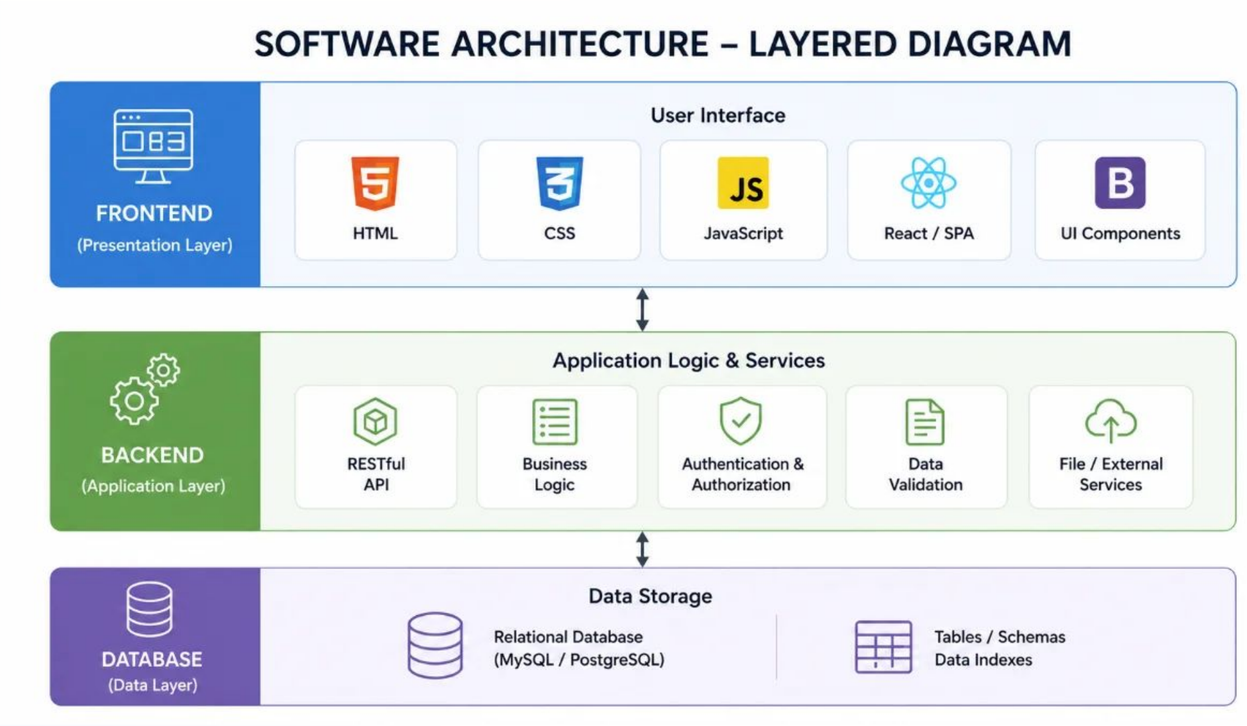
Thực hành

- Khung ứng dụng FastAPI
- Bảo mật mã nguồn với .env
- Tích hợp Gemini Flash
- Kiểm thử endpoint



Vị trí trong dự án xuyên suốt

Buổi 3 là bước ứng dụng bắt đầu "phình to": từ những script rời rạc trở thành một dịch vụ API thực thụ.



Trục thời gian khoá học - khối "Buổi 3" được tô nổi bật.

Mô hình Client và Server

Client

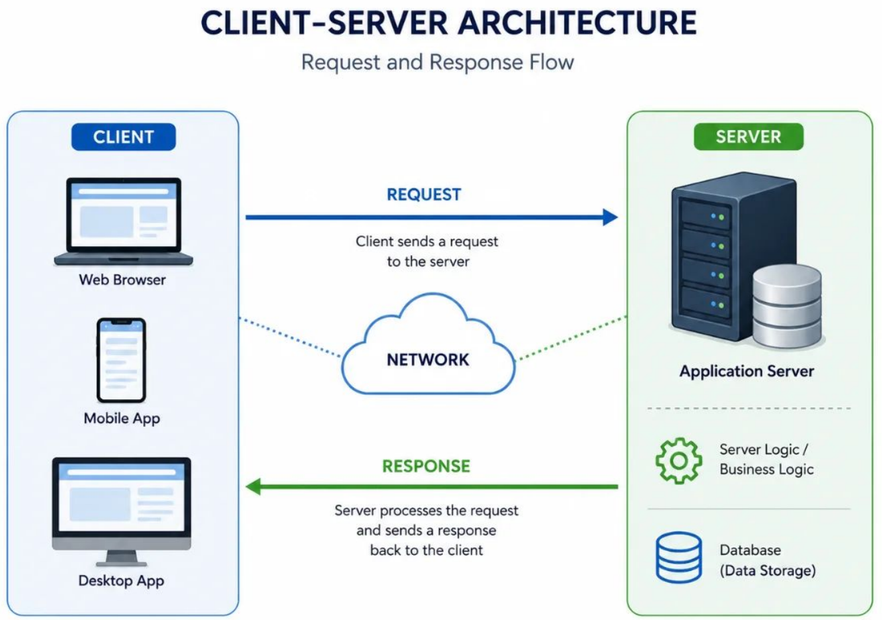
Bên khởi tạo yêu cầu - trình duyệt, ứng dụng di động, hoặc một script Python.

Server

Bên tiếp nhận, xử lý nghiệp vụ và trả về kết quả.

Trao đổi

Hai bên giao tiếp qua cặp **Request** (gửi đi) và **Response** (nhận về).

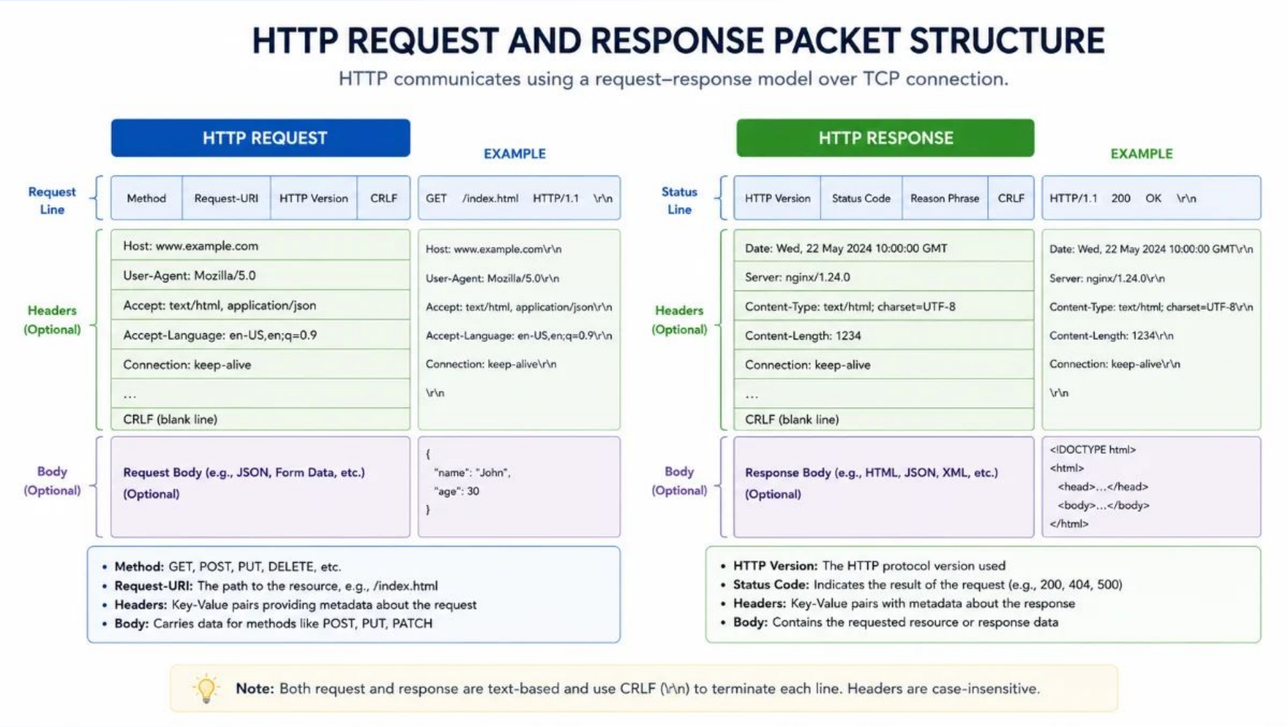


Sơ đồ tương tác Client ⇄ Server.

Cấu trúc một HTTP Request và Response

Header Method, URL, Status Code, thông tin xác thực…

Body Phần dữ liệu đi kèm, thường ở dạng JSON.



Bóc tách cấu trúc một gói tin HTTP: phần Header và phần Body.

Status Code

Ba nhóm mã trạng thái thường gặp - ghi nhớ theo quy tắc đèn giao thông.

ĐẦU MÃ	Ý NGHĨA	VÍ DỤ THỰC TẾ
2xx	**Thành công**	**200 OK**
4xx	**Lỗi phía Client**	**401** Unauthorized (sai API Key) · **404** Not Found
5xx	**Lỗi phía Server**	**500** Internal Server Error (code crash)

REST API và Endpoint

REST API tổ chức dịch vụ quanh các "tài nguyên". Mỗi tài nguyên có một địa chỉ - gọi là **endpoint** - và được thao tác qua các phương thức HTTP (GET, POST…).

`/chat`	Gửi tin nhắn tới mô hình và nhận câu trả lời
`/users`	Quản lý thông tin người dùng
`/health`	Kiểm tra tình trạng hoạt động của dịch vụ

Định dạng dữ liệu JSON

- **JSON Request**

```
POST /chat {

"message": "Xin chào, bạn là ai?" }
```

- **JSON Response**

```
200 OK {

"reply": "Mình là trợ lý AI." ,

"model": "gemini-flash" }
```

FastAPI là gì

01 **Hiệu năng cao**
Xử lý bất đồng bộ (async), tốc độ gần bằng Node.js và Go.

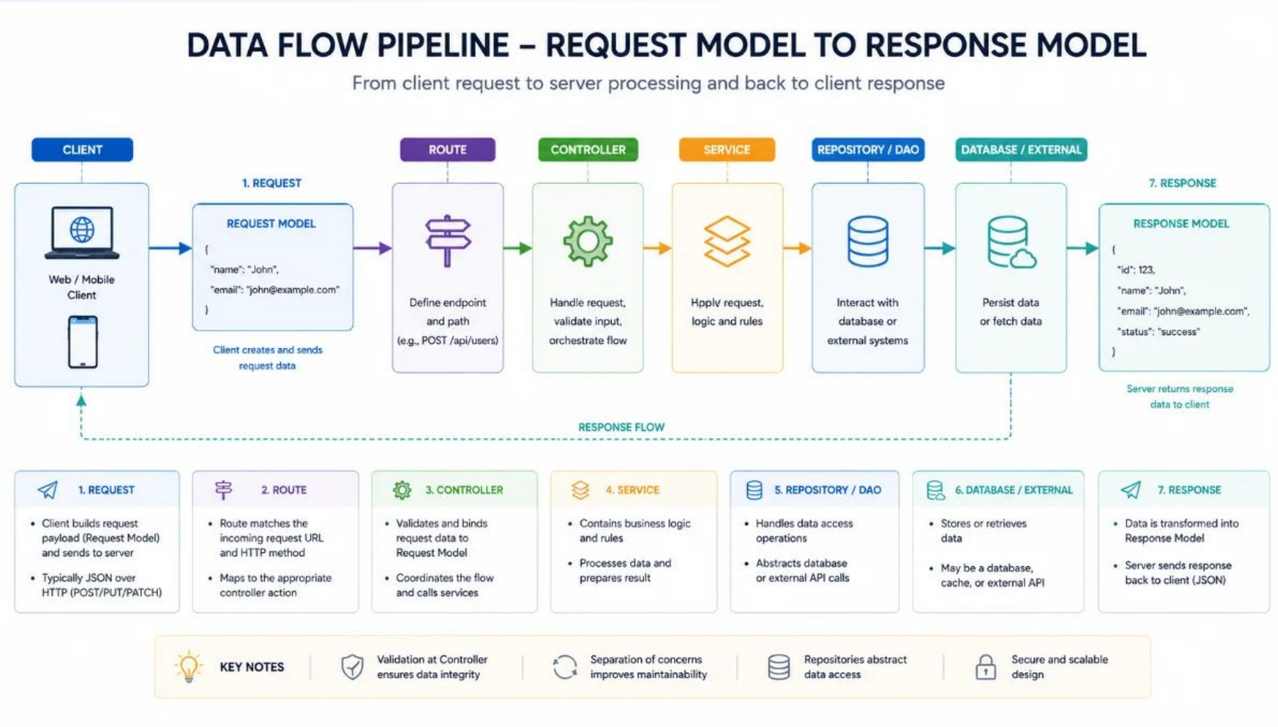
02 **Tự động sinh tài liệu**
Giao diện thử API sẵn tại `/docs` - không cần viết thêm.



Logo chính thức của FastAPI.

Ba thành phần cần biết trong khung ứng dụng mẫu

Một endpoint được ghép từ ba mảnh, chạy theo một đường ống dữ liệu (pipeline).



Request Model (dữ liệu vào) → **Route** (xử lý) → **Response Model** (dữ liệu ra)

Ví dụ một Route cho endpoint /chat

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    answer = call_llm(req.message)
    return ChatResponse(reply=answer)
```

→ **BaseModel**
Tự động kiểm tra dữ liệu đầu vào.

→ **@app.post("/chat")**
Khai báo endpoint nhận yêu cầu POST.

→ **response_model**
Chuẩn hoá dữ liệu trả về đúng định dạng.

CẢNH BÁO BẢO MẬT

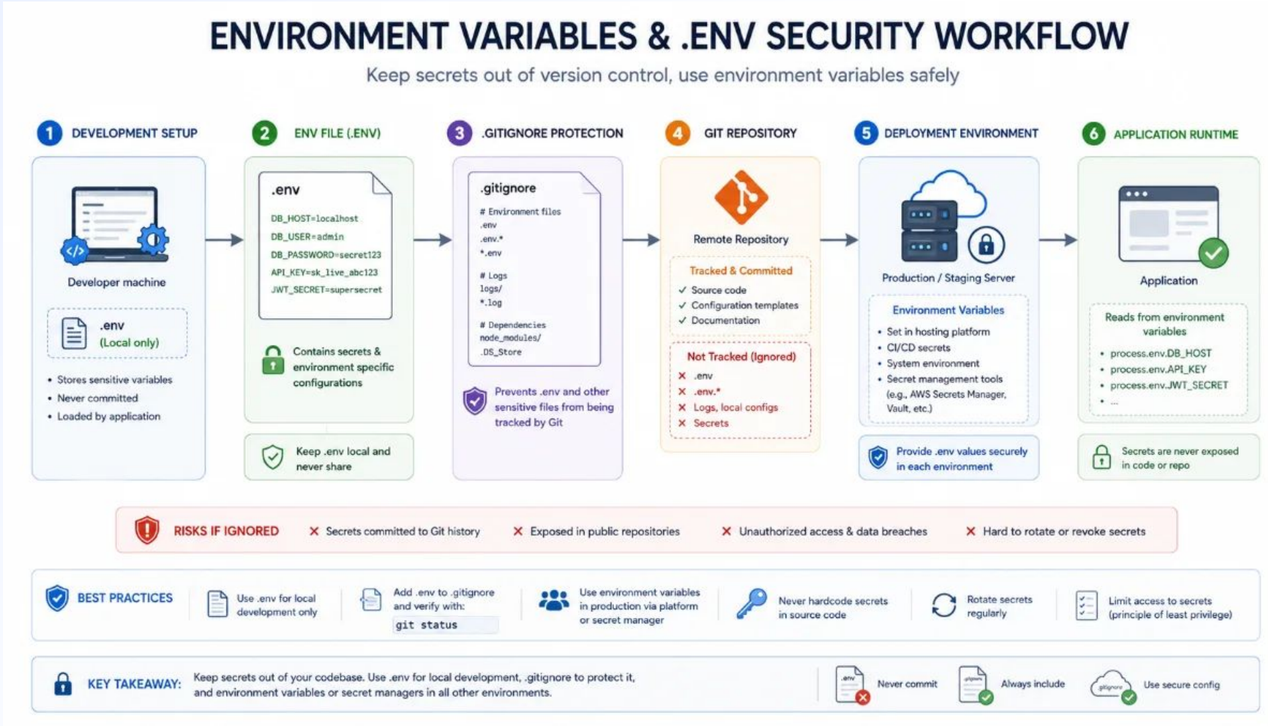
Vì sao không ghi API key thẳng vào mã nguồn?

Đẩy code lên GitHub là API key bị lộ **công khai** . Bất kỳ ai cũng có thể lấy khoá, gọi API và làm bạn cạn hạn mức - hoặc mất tiền.

Không bao giờ hard-code API key.

Biến môi trường và tập tin .env

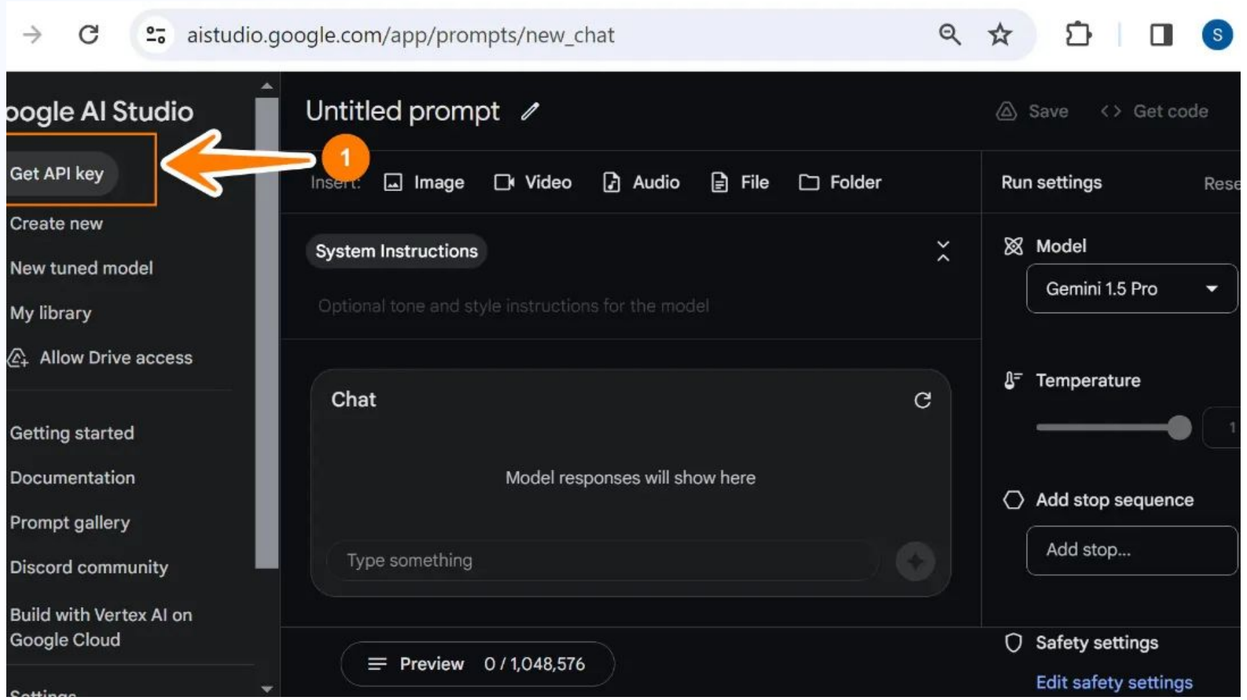
Tách khoá bí mật ra khỏi mã nguồn: đặt trong `.env` ở máy local, và để `.gitignore` chặn nó khỏi Git.



`.env` (local) → **.gitignore** chặn lại → không lên GitHub → an toàn

Google AI Studio và Gemini Flash

1. Đăng nhập **Google AI Studio**.
2. Chọn nút **“Get API key”**.
3. Sao chép khoá vào tập tin `.env`.



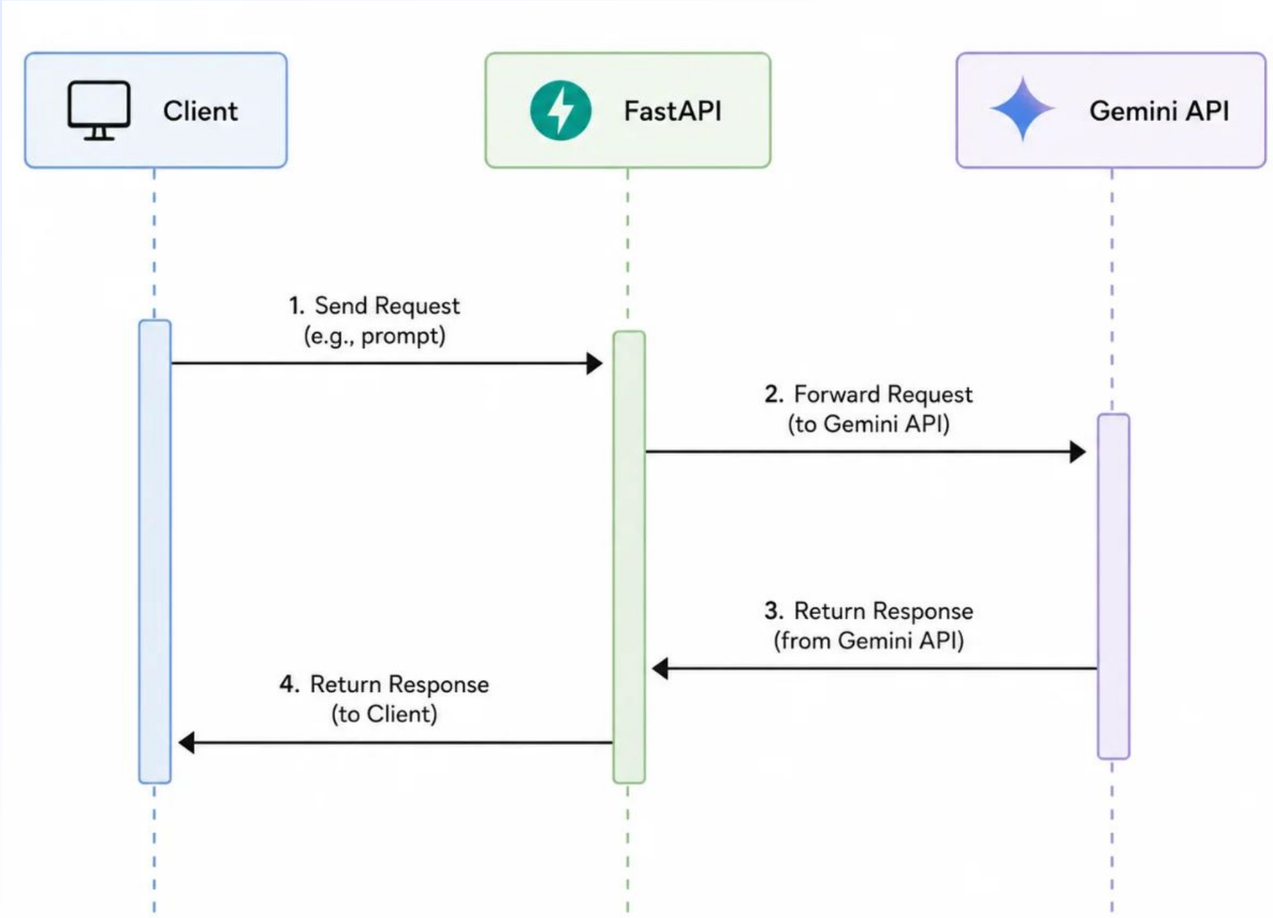
Màn hình tạo API key trong Google AI Studio.

PHẦN E · CỐT LÕI BUỔI HỌC

Luồng gọi LLM

FastAPI đóng hai vai cùng lúc: **vừa là Server** (với Client), **vừa là Client** (với Gemini).

- Client
- Ứng dụng FastAPI (Proxy)
- Gemini API



Sơ đồ tuần tự: Client ⇄ FastAPI ⇄ Gemini API.

Xử lý lỗi khi gọi LLM

401

Sai API Key

Kiểm tra lại khoá trong `.env` ; tạo khoá mới nếu cần.

Timeout

Hết giờ chờ

Đặt thời gian chờ hợp lý và thử lại (retry) khi mạng chậm.

429

Rate Limit

Quá số lượt cho phép; chờ, giảm tần suất hoặc đổi provider.

Phương án dự phòng Groq

IF Gemini gặp lỗi **5xx** hoặc **Rate Limit**

THEN → Đổi API Key và Endpoint sang **Groq**

ELSE → Tiếp tục dùng **Gemini Flash**

Cùng một chuẩn gọi API - chỉ thay khoá và địa chỉ, dịch vụ gần như không gián đoạn.

Kiểm thử bằng curl

bash - localhost

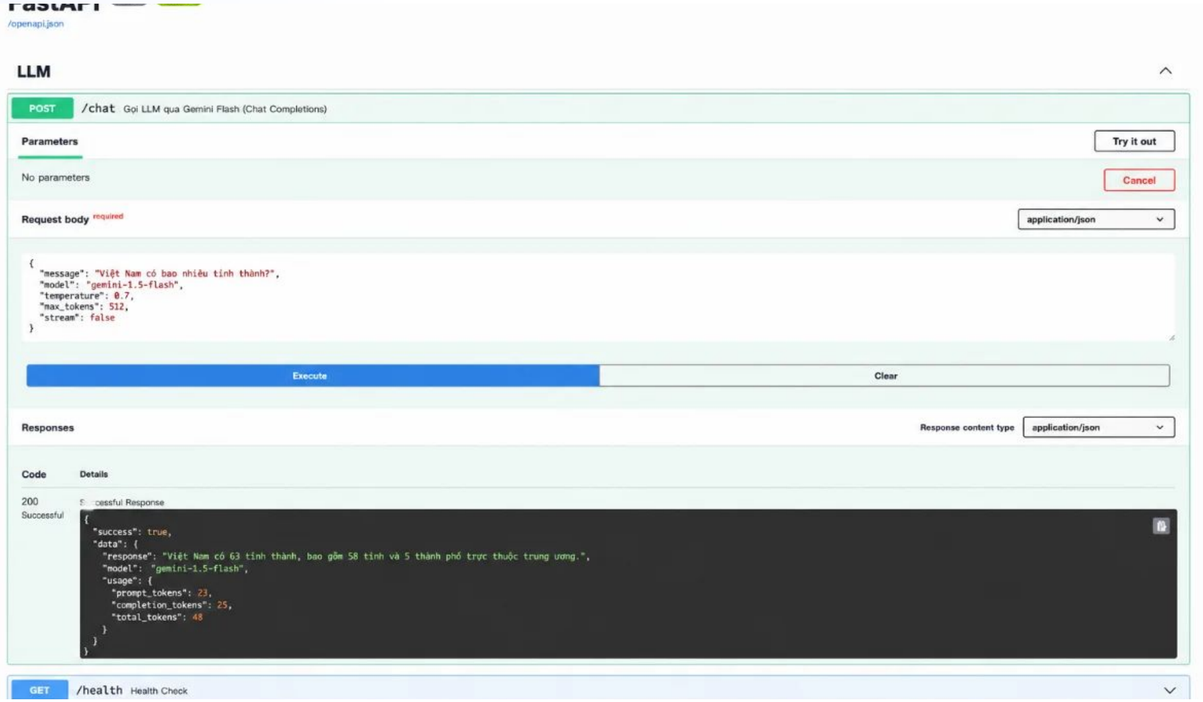
```
$  curl -X POST http://localhost:8000/chat \
    -H
"Content-Type: application/json"  \
    -d
'{"message": "Xin chào"}' {

"reply": "Chào bạn! Tôi có thể giúp gì cho bạn?" ,

"model": "gemini-flash" }
```

Kiểm thử bằng Swagger UI tại /docs

1. Mở `localhost:8000/docs`
2. Chọn endpoint `/chat`
3. Bấm **“Try it out”**, nhập dữ liệu
4. Bấm **“Execute”** và xem Response



Khoanh đỏ: nút “Try it out” · Khoanh vàng: nút “Execute”.